Assessing LLM-Generated Nextflow Workflows

To evaluate the extent to which LLMs can assist in developing workflows using Nextflow, we curate a diverse set of representative workflows from the nf-core framework. The selected workflows span multiple domains, including genomics, transcriptomics, epigenomics, and regulatory genomics, thereby enabling a comprehensive and domain-balanced assessment. The *nf-core/demo* pipeline serves as a foundational example, involving straightforward quality control and adapter trimming of single- or paired-end FASTQ reads, making it useful for testing basic workflow generation capabilities. *Fetchngs* introduces more advanced functionality by automating the retrieval of sequencing data from public repositories, challenging models to reason about metadata management, remote data access, and parsing of structured sample sheets. *Rnaseq* and *methylseq* represent complex, end-to-end pipelines that integrate numerous modules, manage intricate dependencies, and involve advanced data transformations, ideal for evaluating the models' ability to recommend appropriate toolchains, configurations, and parameters. The inclusion of *phyloplace* extends the evaluation into the domain of phylogenetic analysis, specifically testing the models' ability to handle workflows that map query sequences onto a reference phylogenetic tree using specialized inputs and classification tools. These workflows offer a robust and multifaceted benchmark suite to assess how effectively LLMs understand and generate Nextflow-based workflows.

The ***nf-core/demo*** workflow is designed to perform a basic bioinformatics data processing task using a minimal dataset. First, it conducts quality control of raw FASTQ files using the module *FastQC* to assess sequence quality metrics. Next, it applies module *Seqtk* to trim adapter sequences and low-quality bases, improving the quality of the reads. Finally, it aggregates the quality control results using *MultiQC*, generating a summary report that provides an overview of the data quality before and after trimming. We evaluate the workflow using a sample sheet that includes both single-end and paired-end data, available at this link. The workflow is executed using the following command:

```
nextflow run nf-core/demo -profile docker,test
--outdir 'demo-results'
```

We do not delve into the syntax details here, as our primary objective is not to teach Nextflow but to assess whether LLMs can effectively assist in workflow development. Interested readers may consult the official documentation for further guidance. For our purpose, we simply run the workflow on the provided sample dataset and inspect the results. We obtain several output reports from the workflow execution, including FastQC and MultiQC quality control summaries, the execution report, and the pipeline DAG visualization. After verifying these outputs, we proceed to test the same workflow development task using LLMs.

We begin by applying the instruction-only prompt to GPT-4o. The model generates a logically structured set of workflow steps, starting with initial quality control using the *FastQC* module, followed by adapter and quality trimming using tools such as *Trim Galore!*, *fastp*, or *Trimmomatic*. It then suggests performing a second round of quality control to validate the trimming process and concludes with summarizing the results using *MultiQC*. These steps are largely consistent with those in the official nf-core/demo workflow. The primary divergence lies in the trimming step: while the nf-core/demo pipeline uses *seqtk*, GPT-4o proposes alternative tools that are widely accepted and functionally equivalent within the context of short-read preprocessing. Overall, GPT-4o demonstrates a strong ability to generate a correct and coherent workflow based solely

on an instruction-only prompt, indicating a solid understanding of standard practices in NGS data preprocessing with Nextflow.

We then provide the same instruction-only prompt to *Gemini 2.5 Flash*. The model suggests a detailed and modular workflow design. The proposed steps begin with parsing the sample sheet to extract FASTQ file paths using Nextflow channels. It then performs an initial quality assessment using the *FastQC* module, followed by adapter and quality trimming with either *seqtk* or *Trim Galore!*. This is followed by a second round of quality control using *FastQC* on the trimmed reads, and the final step involves aggregating all quality control outputs into a comprehensive report using *MultiQC*. While this plan largely mirrors the nf-core/demo workflow, Gemini extends it by emphasizing modularity and offering both *seqtk* and *Trim Galore!* (which wraps Cutadapt) as valid trimming options. Overall, Gemini 2.5 Flash demonstrates a clear understanding of the workflow structure and appropriately maps tools to each processing stage, showing strong competence in workflow planning from an instruction-only prompt.

We also evaluate the response from DeepSeek-V3 using the same instruction-only prompt. The model provides a well-structured and modular workflow outline for FASTQ quality assessment and preprocessing that is closely aligned with nf-core conventions. The workflow begins with parsing the input sample sheet, followed by initial quality control using *FastQC*, adapter trimming via *Cutadapt*, and then a separate quality filtering step using *FastP*. A second round of quality control is performed post-filtering using *FastQC*, and finally, *MultiQC* is used to generate an aggregated summary report. The model explicitly maps each step to specific nf-core modules and provides a clear implementation structure in *main.nf*, along with a sample configuration in *nextflow.config*.

Among the three models, DeepSeek-V3 provides the most comprehensive and implementation-ready response, outlining a modular workflow with nf-core modules, configuration details, and code structure. Gemini 2.5 Flash also performs well, offering accurate steps with moderate implementation guidance and alignment with nf-core practices. In contrast, GPT-4o delivers a concise and correct high-level outline but lacks the depth needed for direct workflow deployment. Overall, DeepSeek-V3 stands out as the most effective model for building fully functional, production-ready workflow based on instruction-only prompts.

Our second workflow, *nf-core/fetchngs*, is designed to automate the retrieval and preparation of sequencing data from major public repositories including *ENA, SRA, DDBJ, and GEO*. Given a list of accession identifiers, the pipeline retrieves associated metadata and downloads the corresponding raw FASTQ files via *FTP, Aspera, or SRA tools*. It then compiles a standardized samplesheet and organizes the output in a format that is immediately compatible with downstream nf-core pipelines such as *RNA-seq or ATAC-seq*. This automation streamlines the data acquisition process, enhances reproducibility, and reduces the manual burden of managing large-scale sequencing datasets.

To assess LLMs' ability to support the development of such workflows, we first execute the *nf-core/fetchngs* pipeline using a test CSV file containing sample IDs, confirming that it successfully fetches both FASTQ files and metadata. We then provide GPT-4o with an instruction-only prompt to generate the workflow. The output from GPT-4o closely aligns with the core logic of the fetchngs pipeline: parsing sample IDs from a CSV, retrieving metadata, downloading sequencing files, and organizing outputs into structured directories. However, GPT-4o demonstrates greater flexibility by suggesting the integration of additional tools, such as pysradb utilities, which are particularly useful for SRA or GEO-specific workflows. GPT-4o also suggests the quality checking of the FAST files. This flexibility, while advantageous, also necessitates more careful implementation,

especially in resolving ID types, managing errors, and ensuring reproducible execution through proper containerization.

We then provide the same instruction-only prompt to *Gemini 2.5 Flash*. The generated workflow for fetching NGS data is notably detailed and architecturally robust, closely aligning with the nf-core fetchngs pipeline in both structure and objective. Like fetchngs, it accepts a CSV of public sample IDs (e.g., SRA, ENA, GEO, DDBJ), retrieves metadata using the ENA API, and downloads FASTQ files using both direct FTP methods and SRA tools as a fallback. Both workflows include metadata collation and output organization steps. However, Gemini's generated design adds granularity by modularizing each logical step, e.g., separating *download_fastq_direct* and *download_fastq_sra*, validating MD5 checksums, and producing a final samplesheet.csv, and assigning clear containerized environments with resource constraints for each module.

One advantage of the Gemini-generated workflow is its explicit fallback mechanism: if direct FASTQ downloads fail, it reroutes to SRA tools (prefetch + fastq-dump) selectively. Additionally, the inclusion of detailed validation, resource configuration, and Python-based logic for metadata parsing shows an advanced awareness of real-world edge cases (e.g., batch API querying, empty downloads, ENA metadata field inconsistencies). Compared to fetchngs, which abstracts many steps behind a high-level interface, Gemini's approach makes the internals transparent, facilitating educational or research-focused exploration. In the context of evaluating LLM capabilities, Gemini 2.5 Flash demonstrates strong potential in guiding sophisticated workflow design, though its correctness and maintainability would benefit from cross-validation against established pipelines like fetchngs.

We then give the same prompt to the DeepSeek-V3. The generated workflow shares strong structural alignment with the nf-core fetchngs pipeline in retrieving metadata and FASTQ files from public repositories using a list of accessions. DeepSeek-V3 begins by parsing a CSV input, fetching metadata using ENA or SRA APIs, downloading FASTQ files (via enaDataGet or fasterq-dump), and organizing outputs. However, it diverges in its emphasis on user-defined metadata (project, condition) and its incorporation of flexible tuple-based channels to carry metadata across modules. This offers a lightweight yet extensible design. Its use of tuple-passing enables downstream modules (like organize_output) to rename files and structure directories using custom project metadata, a flexibility fetchngs does not natively offer. It also outlines future-proof enhancements like MD5 validation, cloud output integration, and notifications, suggesting practical awareness of real-world deployment. However, compared to the rigorously tested fetchngs, the DeepSeek-V3 workflow requires more manual scripting and error handling, particularly around metadata inconsistencies and download failures. It also lacks built-in support for containerized environments, dependency management, and profiling, which are strengths of nf-core workflows. Overall, DeepSeek-V3 demonstrates strong conceptual competence in building modular, extensible workflows and offers a solid foundation for assessing LLM capabilities in workflow construction.

The workflow generated by GPT-4o, Gemini 2.5 Flash, and DeepSeek-V3 for W2: nf-core/fetchngs*, all demonstrate the capability of LLMs to assist in workflow development, particularly in automating metadata retrieval and FASTQ file downloads from public sequencing databases. GPT-4o provides a concise, modular breakdown that aligns well with nf-core's fetchngs, while also offering flexibility through tool alternatives like pysradb and Entrez utilities. Gemini 2.5 Flash delivers the most production-ready solution, featuring a fully containerized, resource-configured, and modular DSL2 workflow that includes fallback*

strategies, MD5 validation, and samplesheet generation, closely mirroring the robustness of nf-core standards. DeepSeek-V3 emphasizes conceptual modularity with user-defined metadata propagation, conditional tool logic, and practical extensibility for cloud and notification integration, though it lacks some execution-level rigor. While all three LLMs show potential, Gemini 2.5 Flash outperforms the others in completeness, reproducibility, and implementation detail, making it particularly well-suited for generating executable, real-world workflows.

We then move to our next workflow *nf-core/methylseq* for comprehensive analysis of bisulfite-converted sequencing (BS-seq) data to facilitate DNA methylation profiling. The workflow automates key tasks such as raw read quality assessment with *FastQC*, adapter trimming using *Trim Galore!*, and alignment to a reference genome via either *Bismark (with Bowtie2 or HISAT2)* or the *bwa-meth* and *MethylDackel* toolchain, with optional GPU acceleration supported through *Parabricks*. In our implementation, we utilize the Bismark-based workflow. The pipeline further includes deduplication of aligned reads, extraction of cytosine methylation metrics, and comprehensive quality evaluations using *Qualimap, Preseq, and HS* Metrics. Outputs include deduplicated BAM files, detailed methylation calls, and bias reports, all of which are consolidated into a single MultiQC report for streamlined visualization. Additionally, the pipeline generates extensive provenance metadata, including execution logs, software versions, and configuration parameters, ensuring full reproducibility across computing environments.

To evaluate LLM support for constructing this workflow, we first test GPT-4o using an instruction-only prompt. We observe that the workflow generated by GPT-4o exhibits several inaccuracies while suggesting modules, even though the overall sequence of analytical steps was conceptually sound. For example, the model suggests a module called *fastqc_raw* for raw read quality control, which does not exist in the official Nextflow or nf-core module repositories; the correct module name is simply *fastqc*. Similarly, for adapter and quality trimming, the model proposes *trim_reads*, whereas the appropriate module is *trimgalore*. These naming inconsistencies are found throughout the generated workflow, suggesting that the model's responses are not grounded in the actual set of available Nextflow modules. Consequently, we adopt a role-based prompting approach to improve the specificity and correctness of the module recommendations.

We evaluate the workflow produced by GPT-4o using the role-based prompt. It correctly captures the high-level stages of DNA methylation analysis using bisulfite-converted sequencing (BS-seq) data, beginning with FASTQ quality control and adapter trimming, followed by bisulfite-aware alignment, deduplication, methylation extraction, and MultiQC-based reporting. Importantly, the model successfully identified the correct nf-core module names such as fastqc, trim_galore, bismark/genomeprepare, bismark/align, bismark/deduplicate, and bismark/methylationextractor, indicating that role-based prompting can enhance grounding in actual Nextflow components. However, the generated workflow also exhibits several limitations. First, it does not include configurable options for alternative aligners (e.g., bismark_hisat) or GPU-accelerated execution. Second, while the model included key optional QC modules like qualimap, preseq, and picard/collecthsmetrics, it does not explain when or why to use them, missing opportunities to contextualize their role in specific experimental designs (e.g., capture-based methylation assays). Third, the model omitted support for protocol-specific presets such as RRBS (–rrbs) and PBAT (–pbat), which are natively handled in the nf-core implementation. Finally, although the core logic was structurally correct, some processes like CSV parsing are abstracted as custom steps rather than referencing existing solutions.

To assess the capabilities of Gemini 2.5 Flash in constructing the workflow, we evaluate its response to an instruction-only prompt. The model successfully outlines all necessary stages of the workflow, including CSV-based input parsing, raw read quality control, adapter trimming, bisulfite-aware alignment, PCR duplicate removal, methylation extraction, and comprehensive quality control reporting. It recommends appropriate and widely used tools such as *FastQC, Trim Galore!, Bismark, MethylDackel, Qualimap, and MultiQC*, and its modular structure reflects a good understanding of Nextflow DSL2 practices, including containerization, channel dependencies, and resource management. Furthermore, the workflow is logically organized and aligns well with the stages implemented in the official nf-core/methylseq pipeline. However, Gemini shows some limitations. It does not fully specify the conditional logic or flag handling needed to toggle between these options, nor does it elaborate on the inclusion of protocol-specific parameters. Despite these minor gaps, Gemini 2.5 Flash demonstrates strong proficiency in designing structurally sound and tool-appropriate BS-seq workflows, making it a capable model for assisting in the development of complex, real-world pipelines.

We then, evaluate DeepSeek-V3 for its ability to generate a Nextflow workflow for bisulfite-converted sequencing (BS-seq) data analysis using the same instruction-only prompt. The model successfully outlines the key stages of a BS-seq workflow, including input parsing, quality control with FastQC, adapter, and base trimming using Trim Galore!, bisulfite-aware alignment with Bismark or bwa-meth, deduplication, methylation extraction, and downstream quality reporting with MultiQC. The response is structured, includes example CSV formats, command-line syntax for each tool, and even proposed a skeleton main.nf script and directory structure. This indicates that DeepSeek-V3 has a solid conceptual grasp of both BS-seq data analysis and the Nextflow DSL2 design pattern. However, the workflow generated by DeepSeek-V3 also exhibits critical shortcomings. While it named commonly used tools, workflow steps like FASTQC, TRIM_GALORE, or BISMARK are presented as abstract module calls rather than as references to actual modules in the nf-core/modules repository, limiting reproducibility. Additionally, the pipeline lacks any reference to containerization profiles, resource management, or error handling, all critical for robust, production-ready deployment. However, the generated steps are correct, and we can develop the workflow following the instructions.

In comparing GPT-4o, Gemini 2.5 Flash, and DeepSeek-V3 for generating a Nextflow workflow for DNA methylation analysis using bisulfite-converted sequencing (BS-seq) data, we uncover notable differences in how each model responds to different prompting strategies. Under instruction-only prompting, Gemini 2.5 Flash produces the most accessible and beginner-friendly output. It delivers logically structured steps, accurate tool selection, and detailed descriptions suitable for users with limited workflow development experience. DeepSeek-V3 also performed well under instruction-only prompting, offering a clear conceptual outline along with practical elements such as directory structure, example CSV input, and bash commands. This makes it particularly valuable for intermediate users who seek both clarity and low-level control. In contrast, GPT-4o initially underperforms under instruction-only prompting, often hallucinating incorrect module names (e.g., fastqc_raw, trim_reads) and tool references. However, when guided with a role-based prompt, GPT-4o produces the most technically precise and nf–core–compliant workflow, correctly referencing official modules such as fastqc, trim_galore, and bismark/align. In summary, Gemini is best positioned for novice users, followed by DeepSeek-V3 for intermediate users, while GPT-4o offers the highest accuracy and implementation fidelity for

advanced users through prompt specialization.

Our next workflow, *nf-core/rnaseq*, processes raw RNA-seq data into high-quality, analysis-ready outputs. The workflow begins with raw FASTQ files and a metadata samplesheet, and takes a reference genome (FASTA) and annotation file (GTF). It includes modules for inferring strand specificity, performing quality checks (e.g., FastQC, MultiQC), trimming adapters (Trim Galore!), filtering contaminants (BBSplit), removing rRNA (SortMeRNA), and aligning reads using STAR or HISAT2. Quantification is performed using Salmon, RSEM, or StringTie, with optional pseudoalignment via Salmon or Kallisto. The pipeline also supports UMI handling, duplicate marking, transcript assembly, and generation of coverage files. Extensive quality metrics are collected using tools such as *RSeQC, Qualimap, Preseq, and dupRadar*. All results are summarized in an interactive MultiQC report. To evaluate the ability of LLMs to generate this complex workflow, we provide an instruction-only prompt and assess whether the models can accurately develop *rnaseq* pipeline.

We observe that the GPT-4o-generated workflow effectively covers the core analytical stages of RNA-seq data processing, including quality control, adapter trimming, genome alignment, gene/transcript quantification, and differential expression analysis. It accepts a CSV sample sheet with FASTQ links and reference genome inputs, making it flexible and suitable for standard analyses. However, the GPT-4o response has several limitations. The nf-core workflow includes a wider range of features such as pseudoalignment options (e.g., Salmon and Kallisto), UMI handling, strandness autodetection, biotype-level QC, spike-in normalization, and automated GTF/GFF validation, all of which enhance robustness and reproducibility. Moreover, nf-core pipelines benefit from strict input schema validation, containerization, continuous integration testing, and broad community support, making them more reliable for large-scale or production-grade use. The absence of these advanced features in the custom workflow may not affect small or focused studies but could introduce reproducibility challenges and limit flexibility in more complex experimental designs. However, the steps we obtain are fully functional.

We evaluate the output of Gemini 2.5 Flash for constructing a Nextflow-based RNA-seq workflow using an instruction-only prompt. The generated workflow successfully captures all key analytical stages, ranging from sample sheet parsing and raw read quality control to adapter trimming, reference genome indexing, alignment or pseudo-alignment, transcript quantification, differential expression analysis, and quality reporting, demonstrating strong structural alignment with the nf-core/rnaseq pipeline. Gemini provides flexibility in tool selection, supporting both traditional aligners (e.g., STAR) and pseudo-aligners (e.g., Salmon, Kallisto), and adopts modular design principles using channels and containers, consistent with best practices in DSL2-based Nextflow development. However, the Gemini-generated workflow lacks some infrastructure features, such as automated strandedness detection, UMI processing, GTF validation, biotype-level QC summaries, and reproducibility guarantees via CI testing and schema validation. Moreover, while it mentions statistical tools for differential expression (e.g., DESeq2), it does not detail complex contrast designs or robust metadata handling, which are integral to nf-core. As a result, although Gemini provides a conceptually complete and interpretable scaffold suitable for developing the workflow, it can be improved.

We further evaluate the performance of DeepSeek-V3 using the same instruction-only prompt. The RNA-seq workflow generated by DeepSeek-V3 presents a clear, methodical structure that encompasses all essential stages of RNA-seq data processing, starting from sample sheet parsing and reference genome indexing to quality control, adapter trim-

ming, alignment, quantification, differential expression analysis, and final reporting. Its design closely mirrors the nf-core/rnaseq workflow in terms of both modularity and core functionality. DeepSeek appropriately suggests widely adopted tools such as STAR and HISAT2 for alignment, Salmon for quantification, and FastQC, Qualimap, and MultiQC for quality assessment. In addition, the model provides practical implementation notes, including the use of *main.nf, modules.config, and nextflow.config*, which enhance its utility for users with experience in pipeline development. However, it lacks some capabilities integrated into nf-core, such as automated strandness detection, UMI handling, TPM/FPKM normalization options, comprehensive biotype-specific QC metrics, and rigorous input schema validation. DeepSeek also includes practical implementation notes using main.nf, modules.config, and nextflow.config. DeepSeek's generated steps are methodologically sound and capture key conceptual elements of a robust RNA-seq workflow, making it suitable for users familiar with pipeline customization.

The RNA-seq workflow outputs generated by GPT-4o, Gemini 2.5 Flash, and DeepSeek-V3 all cover the essential analytical stages, quality control, trimming, alignment, quantification, and differential expression analysis but differ significantly in depth, infrastructure awareness, and production readiness. GPT-4o provides a concise and methodologically correct outline but lacks some features such as pseudo-alignment options, detailed quality control tools, and workflow execution infrastructure, making it the least complete among the three. Gemini 2.5 Flash offers broader tool coverage, supporting both alignment-based and alignment-free methods, and presents a modular structure that is conceptually clear and well-suited for educational use; however, it stops short of providing implementation details like Nextflow process logic or configuration scaffolding. In contrast, DeepSeek-V3 delivers the most comprehensive and technically mature output, it not only includes accurate tool suggestions and optional enhancements (e.g., rMATS, ComBat) but also outlines concrete Nextflow components (main.nf, modules.config, nextflow.config), container usage, and execution profiles, closely resembling the structure and flexibility of the nf-core/rnaseq pipeline. Thus, DeepSeek-V3 outperforms the others in terms of completeness and deployability, followed by Gemini for clarity and breadth, with GPT-4o being the most minimal in practical usability.

We conclude our evaluation with the *nf-core/phyloplace* workflow, which is designed to facilitate efficient and reproducible phylogenetic placement of query sequences onto a reference tree. The workflow operates in two distinct modes: (i) placement-only, where input sequences are directly aligned to a reference multiple sequence alignment and placed onto a fixed phylogenetic tree using likelihood-based methods; and (ii) search-plus-placement, where input sequences from a larger, unfiltered dataset are first screened using profile-based searches to identify candidates for subsequent alignment and placement. After placement, the workflow grafts the query sequences onto the reference tree and performs taxonomic classification, generating annotated phylogenies, placement summaries, and visualizations. Developed using Nextflow DSL2 and fully containerized modules, *nf-core/phyloplace* ensures portability, scalability, and reproducibility across diverse computing environments. We execute the workflow following the instructions of *nf-core/phyloplace*. The workflow generates a comprehensive set of output files that summarize the results of the phylogenetic placement and classification process. The workflow produces several key output files organized per sample. These include: (i) placement files containing likelihood-based placements of query sequences on the reference tree; (ii) grafted phylogenetic trees in Newick format with query sequences inserted into the reference topology; (iii) classification tables summarizing the taxonomic or user-defined

annotation of each placed sequence; (iv) sequence alignments showing how queries are aligned to the reference multiple sequence alignment; and (v) visualizations and summary reports, such as heat trees and placement statistics, generated via automated reporting tools. A final MultiQC report aggregates log and quality-control metrics across all samples. These outputs facilitate interpretation, visualization, and downstream analysis of the phylogenetic context of newly placed sequences.

To evaluate the ability of LLMs to assist in developing the *nf-core/phyloplace* workflow, we begin by providing an instruction-only prompt to GPT-4o. The resulting workflow aligns closely with the official nf-core/phyloplace implementation in terms of structure, logic, and tool selection. GPT-4o successfully captures both operational modes, placement-only and search-plus-placement, and accurately identifies the required input components, such as query FASTA files, reference alignments, rooted phylogenetic trees, evolutionary models, HMM profiles, and optional taxonomy files. The model outlines key steps, including candidate filtering using *hmmsearch*, sequence alignment via *mafft*, phylogenetic placement using *epa-ng*, tree grafting with *gappa*, optional taxonomic classification, and summarization with *MultiQC*. In addition, GPT-4o includes implementation-aware suggestions such as input validation, modular output organization, and execution metadata logging.

We then provide the same instruction-only prompt to Gemini 2.5 Flash. We observe that it accurately captures both operational modes and correctly identifies the necessary input components, including query and reference sequences, phylogenetic tree, evolutionary model, HMM profiles, and optional taxonomy. It organizes the workflow into well-defined functional modules such as sequence alignment, phylogenetic placement, classification, and report generation and correctly associates standard tools like MAFFT, HMMER (hmmsearch), EPA-NG, and GAPPA with each step. Though, it omits some nf-core-specific features such as execution metadata (e.g., DAGs and trace files), the overall structure is technically sound and implementation-ready.

We finally give the same prompt to DeepSeek-V3. It also captures both placement-only and search-plus-placement modes and structures the pipeline into logically ordered modules. It specifies necessary input parameters, including reference alignment, rooted phylogenetic tree, evolutionary model, taxonomy file, and HMM profiles, and introduces validation steps to ensure input integrity. The outlined modules, covering HMM-based sequence filtering (hmmsearch), alignment (MAFFT or HMMER), placement (EPA-NG or pplacer), and classification (gappa or taxit), are consistent with tools used in the official pipeline. DeepSeek-V3 also incorporates optional visualization using tools such as ETE3, and Python, and provides a pseudocode-level implementation using Nextflow syntax.

All three LLMs, successfully reconstruct a modular Nextflow workflow for phylogenetic placement, capturing both operational modes and correctly identifying core steps such as input validation, sequence alignment, phylogenetic placement, classification, and reporting. GPT-4o provides a technically accurate and semantically faithful outline closely aligned with the nf-core implementation. Gemini 2.5 Flash presents a well-organized modular architecture using intuitive labels and standard tools, though it omits some nf-core-specific features like execution metadata. DeepSeek-V3 stands out by offering the most implementation-ready response, including detailed parameter definitions, alternative tools (e.g., pplacer, taxit), and a fully structured Nextflow pseudocode, making it especially valuable for advanced users or developers seeking practical scaffolding. While all three are valid, DeepSeek-V3 provides the most comprehensive and adaptable solution,

GPT-4o ensures the highest fidelity to the reference workflow, and Gemini excels in clarity for novices. Therefore, DeepSeek-V3 is best suited for expert-driven implementation, GPT-4o for documentation reproduction, and Gemini for accessible modular reasoning.

Overall Summary for Nextflow Workflows

In summary, the results demonstrate that LLMs can substantially assist in developing workflows for both Galaxy and Nextflow platforms. Gemini 2.5 Flash consistently outperforms other models for Galaxy workflows by generating accurate, tool-aware, and pedagogically rich workflows, offering the best balance between technical detail and usability. Its outputs are both complete and executable within the Galaxy environment, making it the most effective for supporting workflow design in this system. For Nextflow, DeepSeek-V3 emerges as the most capable model. It generates structurally sound, implementation-ready workflows that align closely with nf-core standards, including modular Nextflow DSL-2 structures, accurate tool mapping, container usage, and configuration files. While GPT-4o and Gemini provide reasonable outputs, they either lack the depth of implementation (GPT-4o) or fall short on infrastructure-level detail and flexibility (Gemini) compared to DeepSeek. Overall, the findings confirm that LLMs can play a transformative role in supporting workflow development, with model performance varying by system and use-case complexity.